

Wavelength

Behavioral Compatibility Dating App — FastAPI Mobile Development Documentation

Product & Engineering Documentation

July 2026

Table of Contents

1. Introduction

- 1.1 Purpose of this Document
- 1.2 Product Summary
- 1.3 Why FastAPI Instead of Firebase-Only

2. Feature Set

- 2.1 Core Mechanic (MVP)
- 2.2 Monetization Features
- 2.3 Explicitly Deferred (Phase 2+) Features

3. Product & Growth Strategy Notes

- 3.1 Cold-Start Strategy
- 3.2 Question Types — Decide Explicitly
- 3.3 Monetization UX Guardrail

4. System Architecture

- 4.1 High-Level Architecture
- 4.2 Recommended Tech Stack
- 4.3 Why WebSockets for Chat, Not Firestore Realtime
- 4.4 Environments
- 4.5 Configuration
- 4.6 Current Repository State (Baseline)

5. Authentication Flow

- 5.1 Current State (Implemented)
- 5.2 Target Production Auth Flow
- 5.3 Auth Sequence Diagram (Login)
- 5.4 Security Notes

6. Monetization: Coin Economy

- 6.1 Why Coins Over Subscription
- 6.2 Starting Balance & Sinks
- 6.3 Coin Bundle Pricing (tiered, not flat-rate)
- 6.4 Apple / Google In-App Purchase Compliance (Mandatory)
- 6.5 Coin Expiry Policy

7. Data Model

- 7.1 Entity Overview
- 7.2 Table: `users` (extends existing model)
- 7.3 Table: `questions`
- 7.4 Table: `user_answers`
- 7.5 Table: `question_ratings`
- 7.6 Table: `compatibility_cache`
- 7.7 Coin & Payment Tables

7.8 Chat Tables

7.9 Trust & Safety Tables

7.10 Misc

8. API Reference (Target Surface)

8.1 Auth (`router/auth.py` , extended)

8.2 Profile

8.3 Questions & Answers

8.4 Matching / Shortlist

8.5 Chat

8.6 Coins & Payments

8.7 Trust & Safety

8.8 Admin (internal use, separately authenticated)

9. Matching Algorithm

9.1 Basic Scoring

9.2 Quality-Weighted Scoring (Phase 2)

9.3 Computation Strategy

9.4 Progressive Teaser Count

10. Mobile App: Screens & User Flows

10.1 Full Screen List

10.2 Primary Navigation Structure

10.3 Key UX Principle

11. Notifications Strategy

12. AI-Generated Question Content Pipeline

12.1 Seeding Strategy

12.2 Generation Prompt Structure (example)

12.3 Quality Feedback Loop

13. Trust & Safety

14. Non-Functional Requirements

15. Development Roadmap

Phase 0 — Current State (done)

Phase 1 — MVP (single-city launch)

Phase 2 — Depth & Retention

Phase 3 — Scale & Sophistication

Phase 4 — Growth & Internationalization

16. Appendix

16.1 Glossary

16.2 Suggested Repository Layout Going Forward

16.3 Key Decisions Log (from product discussion)

1. Introduction

1.1 Purpose of this Document

This document translates the product concept discussed for a behavioral, “would-you-rather”-style dating app into a concrete technical and product specification suitable for building a real, production-ready mobile application. It covers the product features, the backend architecture built on **FastAPI**, the database schema, the authentication flow, the matching algorithm, the coin-based monetization system, and the full set of mobile screens and user flows needed to build the app end to end.

The working name used throughout this document is **Wavelength** (“people on the same wavelength”). This is a placeholder — rename as desired before building brand assets.

1.2 Product Summary

Wavelength is a mobile dating app that replaces photo-swiping with a **behavioral compatibility quiz**. Instead of judging strangers on looks first, users answer a sequence of “would you rather” / situational questions (e.g. “*You find a wallet full of cash and no ID — what do you do?*”). After each question, the pool of people who answered similarly is narrowed down. After 5–10 questions, the user is shown a shortlist of people who think like them, and can start a conversation with them in-app.

1.3 Why FastAPI Instead of Firebase-Only

The original idea was to build this on Firebase alone to move fast without “a backend.” The conclusion reached during product discussion — and the basis for this document — is:

- Firestore is itself a backend-as-a-service; the question is really “*custom backend vs. BaaS*,” not “*backend vs. no backend*.”
- **Compatibility matching is a query pattern Firestore does not do well.** “Find users whose answers overlap with mine by X% across N questions” has no native equivalent in Firestore — it has no joins and no server-side aggregation across documents. Doing this client-side means shipping other users’ answer data to every device (a privacy problem) and doesn’t scale.
- A relational database (PostgreSQL) with **FastAPI** as the application layer lets us compute compatibility with indexed SQL queries and precomputed/cached scores, keeps sensitive data (raw answers) server-side only, and gives full control over the coin ledger, in-app-purchase receipt validation, moderation tooling, and analytics — all of which are awkward or expensive to build purely on client-side Firestore rules.
- FastAPI + PostgreSQL is also the stack already scaffolded in this repository (see §4.6), so this plan builds directly on existing work rather than starting over.

Firebase (or an equivalent) still has a place in this architecture — **not as the database**, but as a push-notification provider (FCM) and optionally for object storage / crash analytics. See §4.

2. Feature Set

2.1 Core Mechanic (MVP)

1. **Sign up / Log in** with email + password (JWT-based session).
2. **Profile setup**: name, date of birth, gender, gender preference(s), city, 2–6 photos, short bio.
3. **Question flow**: user is served a session of 10 questions drawn from a seed bank (starting at 30 questions, growing toward 100+ over time). Each question is multiple-choice (“would you rather A / B”, or a situational scenario with 2–4 outcome options).
4. **Progressive shortlist teaser**: after each answered question, the backend recomputes how many *other users* still overlap with this user’s answers so far, and the app shows a live, growing (or shrinking) count — “14 people think like you so far” — with blurred/silhouette avatars.
5. **Reveal**: once the user finishes the full session (10 questions), the full shortlist is revealed with real names/photos and a compatibility percentage per person, ranked by overlap score.
6. **In-app messaging**: once revealed, the user can start a conversation with anyone on their shortlist by spending 1 coin (see §6). All messaging happens in-app — no external contact info is exchanged.
7. **Question rating**: after answering a question, the user may optionally rate it 1–5 stars. This produces a `quality_score` per question over time (see §7.3), which is used both to curate the bank and to gate a future “answer only popular/highly-rated questions” paid mode.

2.2 Monetization Features

Finalized model: **coin-based micro-payments**, not a subscription — chosen deliberately because usage of a dating app is bursty/curiosity-driven rather than continuous, so a monthly subscription mismatches actual behavior (see §6 for full reasoning and pricing).

Coin-gated actions (“coin sinks”):

#	Action	Cost	Rationale
1	Start a new conversation with a shortlisted match	1 coin	Core sink; matches original idea
2	“Why we matched” compatibility breakdown (which answers aligned)	1 coin	Unique to this product — not offered by Tinder/Bumble
3	Instant / early shortlist reveal (before finishing all 10 questions)	2 coins	Direct implementation of the user’s “on-the-spot check (premium)” idea, offered as an <i>option</i> rather than a hard paywall
4	Profile boost (higher placement in others’ shortlists for 24h)	3 coins	Extra revenue surface, common in dating apps
5	Re-roll / skip to next-best match instead of top match	1 coin	Gives paying users control without blocking free flow

New users start with **3 free coins** (not 1) so that a first session can plausibly result in more than one conversation attempt — see §6.2 for the reasoning.

2.3 Explicitly Deferred (Phase 2+) Features

- Read receipts, typing indicators, media (photo/voice) messages in chat.
- Social login (Google, Sign in with Apple) and phone/OTP verification.

- ML-based re-ranking of matches beyond raw answer overlap (e.g. weighting questions by how discriminating they are, or learning from who a user actually messages/likes).
 - Profile photo moderation via an image-safety API.
 - Multi-city / multi-language expansion tooling.
 - Admin web dashboard for question bank curation and moderation queue.
-

3. Product & Growth Strategy Notes

3.1 Cold-Start Strategy

Dating apps live or die on density: with only 30–100 questions and few users, most people will see near-empty shortlists, which kills first impressions immediately. Recommended approach — mirroring how Tinder and Facebook bootstrapped:

- Launch in **one dense, well-defined population** first (a single university, or a single city district), not broadly.
- Gate signups by city/region at first (a simple `city` field + allow-list is enough for MVP; no need for real geofencing yet).
- Only open a new city once the current one has enough active users to make shortlists non-empty for a typical new signup (track this as an internal metric — see §11).

3.2 Question Types — Decide Explicitly

Two different kinds of questions can go into the bank, and they should be **tagged and tracked separately** because they serve different goals:

- **Direct compatibility questions** — “Do you want kids?”, “Are you a night owl?” — predictive of practical compatibility.
- **Situational / scenario questions** — “You find a wallet full of cash, no ID — what do you do?” — more about values/vibe, more novel and engaging, harder to validate as predictive.

Both types are supported by the same schema (`questions.category`), and the AI question-generation prompts (§7.2) should be told explicitly which type to produce per batch, so the mix stays deliberate rather than accidental.

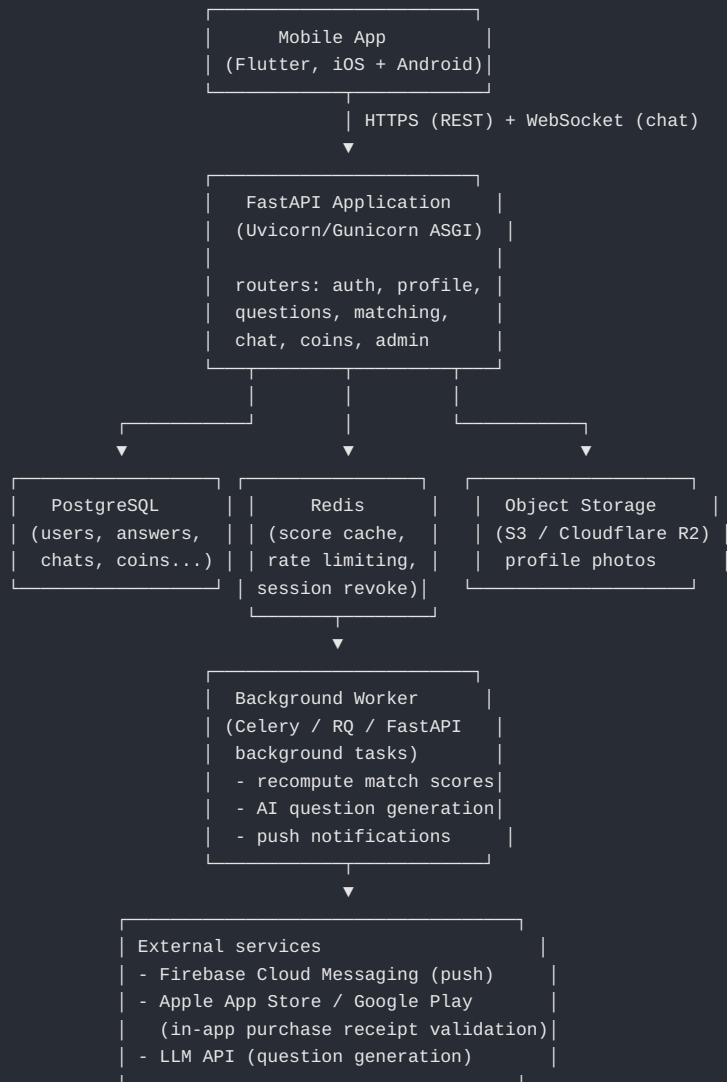
3.3 Monetization UX Guardrail

Avoid the “pay to find out if it was worth it” dark pattern (e.g. showing only a locked count with no faces at all until payment). The design in §2.2 avoids this by:

- Always revealing full profiles (name + photo + %) for free once a user completes the natural 10-question flow — the free tier reaches a real payoff, not a wall.
 - Charging coins only for *acceleration* (instant reveal, unlocking messaging, boosts) and *extra insight* (compatibility breakdown) — value-additive purchases, not “pay to see what you already earned.”
-

4. System Architecture

4.1 High-Level Architecture



4.2 Recommended Tech Stack

Layer	Choice	Notes
Mobile client	Flutter (Dart), single codebase for iOS + Android	Recommended for a solo/small team; React Native is a valid alternative if the team is more JS-native
API framework	FastAPI (already scaffolded in this repo)	Async, typed, auto-generated OpenAPI docs at <code>/docs</code>
Database	PostgreSQL via SQLAlchemy + Alembic migrations	Already partially wired up (<code>app/database.py</code>)
Caching / rate limiting	Redis	Cache hot shortlist computations, enforce login rate limits, store short-lived refresh-token/revocation state
Background jobs	Celery (Redis broker) or FastAPI <code>BackgroundTasks</code> for MVP	Recomputing match scores, AI question generation batches, sending push notifications
Realtime chat	WebSockets (native FastAPI/Starlette support)	Simple long-poll fallback acceptable for MVP
Push notifications	Firebase Cloud Messaging (FCM)	This is the appropriate role for Firebase in this stack — not as the primary datastore
Object storage	S3-compatible storage (AWS S3 or Cloudflare R2)	Profile photos; served via signed URLs or CDN
Auth tokens	JWT (access + refresh), <code>python-jose</code> , <code>passlib[bcrypt]</code>	Already partially implemented (<code>app/auth.py</code>)
In-app purchases	Apple StoreKit + Google Play Billing	Mandatory for selling coins per store policy — see §6.4
AI question generation	Any LLM API (OpenAI/Anthropic/etc.)	Used offline/async to seed and grow the question bank, not called live per-request

4.3 Why WebSockets for Chat, Not Firestore Realtime

Since PostgreSQL is now the system of record, chat messages should be written to Postgres via a FastAPI WebSocket endpoint (or a simple REST `POST /messages` + short-poll for MVP simplicity), and fanned out to the recipient's open WebSocket connection if online, plus a push notification via FCM if offline. This keeps all user data in one place and avoids running two independent databases.

4.4 Environments

- **Local development:** Docker Compose with `api`, `postgres`, `redis` services.
- **Staging:** mirrors production, used for QA and App Store/Play internal testing tracks.
- **Production:** managed Postgres (e.g. RDS/Cloud SQL) + managed Redis + containerized FastAPI behind a load balancer (e.g. on Fly.io, Render, AWS ECS, or similar).

4.5 Configuration

Environment variables (see `.env`, already referenced by `app/database.py` and `app/auth.py`):

```
DATABASE_URL=postgresql://user:password@host:5432/wavelength
SECRET_KEY=<random 256-bit secret, rotate per environment>
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=30
REDIS_URL=redis://host:6379/0
AWS_S3_BUCKET=wavelength-profile-photos
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
FCM_SERVER_KEY=...
APPLE_APP_STORE_SHARED_SECRET=...
GOOGLE_PLAY_SERVICE_ACCOUNT_JSON=...
LLM_API_KEY=...
```

4.6 Current Repository State (Baseline)

The repository already contains a minimal FastAPI skeleton that this plan builds on directly:

- `app/main.py` — FastAPI app instance, router registration, `/health` check.
- `app/database.py` — SQLAlchemy engine/session setup, reads `DATABASE_URL` from `.env`.
- `app/auth.py` — password hashing (`bcrypt` via `passlib`) and JWT creation (`python-jose`).
- `app/user_model.py` — initial `User` SQLAlchemy model (`id`, `email`, `hashed_password`, `is_active`, `language`, `timestamps`).
- `app/schemas.py` — `UserRegister` / `UserLogin` Pydantic schemas.
- `router/auth.py` — `POST /signup`, `POST /login`, `POST /token` (OAuth2-compatible password flow).
- `router/me.py` — `get_current_user` dependency + `GET /user_profile`.
- `router/users_route.py` — `GET /users` (list, for internal/dev use only — should be admin-gated or removed before production).

Everything in §5–§9 below is expressed as an extension of this existing structure (new SQLAlchemy models, new routers, new Pydantic schemas), not a rewrite.

5. Authentication Flow

5.1 Current State (Implemented)

The repo already implements a working baseline:

1. `POST /signup` — validates email + password, hashes the password with `bcrypt`, creates a `User` row, returns basic user info (no token yet — client must call `/login` next).
2. `POST /login` (JSON body) or `POST /token` (OAuth2 `application/x-www-form-urlencoded`, for Swagger UI / OAuth2-compatible tooling) — verifies credentials, issues a signed JWT (`HS256`, 60-minute expiry) containing `sub` (email) and `exp`.
3. `GET /user_profile` — protected route; `get_current_user` dependency decodes the bearer token, loads the `User` row by email, and injects it into the route.

5.2 Target Production Auth Flow

The MVP should extend the existing flow with the following, before public launch:

1. **Access + refresh tokens.** Short-lived access token (15–60 min) + longer-lived refresh token (30 days), so the mobile app can silently refresh sessions instead of forcing re-login. Store a hashed refresh token (or its `jti`) in Postgres/Redis so it can be revoked (logout, password change, account deletion).
2. **Email verification.** On signup, send a verification email/OTP; gate full app access (matching, chat) behind `is_verified = true` to reduce fake accounts.
3. **Rate limiting on `/login` and `/signup`** (via Redis) to prevent brute-force and spam sign-ups.
4. **Password reset flow:** `POST /auth/forgot-password` → emails a time-limited reset token → `POST /auth/reset-password`.
5. **Device/session tracking:** store `push_token` + platform per device so FCM notifications can be targeted, and so a user can see/revoke active sessions.
6. **Account deletion / deactivation** endpoint, required by both app stores' privacy policies.
7. (Phase 2) Social login (Sign in with Apple is mandatory on iOS if any other third-party login, e.g. Google, is offered) and/or phone number + OTP as an alternative to email.

5.3 Auth Sequence Diagram (Login)



5.4 Security Notes

- Never store plaintext passwords — already handled via `passlib/bcrypt`.
- Rotate `SECRET_KEY` per environment; never commit it (already gitignored via `.env`).
- Validate and sanitize all user-generated content (bio, chat messages) server-side.
- Enforce HTTPS everywhere; set secure, httpOnly-equivalent storage on mobile (Keychain/Keystore, not plain `SharedPreferences / UserDefaults`).
- Add per-IP and per-account rate limits on auth and messaging endpoints to reduce abuse/spam.

6. Monetization: Coin Economy

6.1 Why Coins Over Subscription

Usage of a dating app is bursty and curiosity-driven rather than continuous like a media subscription (Netflix/Spotify) — a user might open the app for a week out of curiosity, then not again for a month. A flat monthly subscription mismatches that pattern and depresses conversion. Pay-per-use coins (similar to Coffee Meets Bagel's "beans," and common in Korean/Japanese dating and social apps) align cost with actual usage and lower the psychological barrier to a first purchase.

6.2 Starting Balance & Sinks

- New users start with **3 free coins** (not 1) — enough to attempt a few conversations in the first session, which matters far more for long-term conversion than minimizing the free-coin giveaway. A user who never has one good first conversation churns immediately and never becomes a payer.
- Coin sinks (recap from §2.2): start conversation (1), compatibility breakdown (1), instant reveal (2), profile boost 24h (3), re-roll to next match (1).
- Coins are consumed via a single server-side wallet debit endpoint, always inside a DB transaction, so balance and the action it pays for succeed or fail atomically (see `coin_transactions` in §7.4).

6.3 Coin Bundle Pricing (tiered, not flat-rate)

Flat "\$2 = 1 coin" pricing leaves revenue-per-payer on the table. Use bundles with escalating per-coin value, which nudges buyers toward bigger packs:

Bundle	Price (indicative, USD)	Effective price/coin
3 coins	\$2.99	\$1.00
10 coins	\$7.99	\$0.80
25 coins	\$14.99	\$0.60

Exact prices should be tuned per market and after accounting for store fees (\$6.4); use these ratios as a starting point, and A/B test once there's real traffic.

6.4 Apple / Google In-App Purchase Compliance (Mandatory)

Coins are a virtual currency, so **Apple App Store and Google Play policies require all purchases of coins to go through StoreKit / Google Play Billing** — not a custom Stripe/PayPal flow inside the app. This costs **15–30%** of revenue (15% under the small-business threshold, 30% above, on both platforms as of current policy — confirm current rates before launch). Build this into pricing from day one; do not attempt to route coin purchases around IAP or the app risks store rejection/removal.

Backend responsibilities:

1. Mobile app initiates purchase via StoreKit/Play Billing SDK.
2. On successful purchase, the client sends the **receipt/purchase token** to `POST /coins/purchases/verify`.
3. FastAPI backend verifies the receipt server-side against Apple's App Store Server API / Google Play Developer API (never trust the client-reported coin amount).
4. On successful verification, backend credits the wallet, records an `iap_purchases` row, and returns the updated balance.
5. Handle subscription/purchase restore, refunds, and Play/Apple server-to-server notifications (App Store Server Notifications v2 / Real-time Developer Notifications) to reconcile chargebacks/refunds against the coin ledger.

External payment methods (e.g. JazzCash, EasyPaisa, direct Stripe) can only be used for things that are **not** in-app virtual currency/features — e.g. a physical/real-world perk, if one is ever added.

6.5 Coin Expiry Policy

Deliberate decision (rather than a default): **purchased coins never expire**. Any promotional/bonus coins (e.g. a signup bonus beyond the base 3, referral bonuses, or seasonal promos) expire **30 days** after being granted. This preserves goodwill for money actually paid, while still creating urgency around promotional giveaways.

7. Data Model

7.1 Entity Overview

```
users —< profile_photos
users —< user_answers >— questions —< question_options
users —< question_ratings >— questions
users —< coin_wallet (1:1)
users —< coin_transactions
users —< iap_purchases
users —< conversations >— users (conversation has exactly 2 participants for MVP)
conversations —< messages
users —< reports / blocks >— users
users —< devices
users —< notifications
users —< compatibility_cache >— users (precomputed pairwise scores)
```

7.2 Table: `users` (extends existing model)

Existing columns (already implemented in `app/user_model.py`): `id`, `email`, `hashed_password`, `created_at`, `updated_at`, `is_active`, `language`.

Additional columns needed for the product:

Column	Type	Notes
<code>name</code>	String	Display name
<code>date_of_birth</code>	Date	Used to compute age; enforce 18+ at signup
<code>gender</code>	Enum	e.g. <code>male</code> , <code>female</code> , <code>nonbinary</code> , <code>other</code>
<code>gender_preference</code>	Array/Enum[]	Who this user wants to be shown
<code>city</code>	String (indexed)	Used for cold-start city gating (\$3.1)
<code>bio</code>	Text	Optional short bio
<code>is_verified</code>	Boolean	Email/OTP verified
<code>is_banned</code>	Boolean	Moderation flag
<code>is_onboarded</code>	Boolean	Has completed profile setup + first question session
<code>last_active_at</code>	DateTime	For activity-based ranking/boost decay

7.3 Table: `questions`

Column	Type	Notes
<code>id</code>	PK	
<code>prompt</code>	Text	The situational/behavioral question text
<code>category</code>	Enum	<code>direct_compatibility</code> <code>situational_values</code> (see §3.2)
<code>tags</code>	Array[String]	e.g. ["ambition", "conflict-style"]
<code>is_active</code>	Boolean	Whether it's currently served
<code>avg_rating</code>	Float	Rolling average from <code>question_ratings</code>
<code>rating_count</code>	Integer	
<code>times_answered</code>	Integer	Popularity/exposure tracking
<code>quality_score</code>	Float	Derived score used to rank/curate (§ below)
<code>created_by</code>	Enum	<code>ai_generated</code> <code>manual</code>
<code>created_at</code>	DateTime	

`question_options` (child table, 2–4 rows per question): `id`, `question_id`, `label`, `order_index`.

Quality score combines average rating and “discriminating power” (how evenly the answers split across options — a question everyone answers identically is bad at differentiating people): $\text{quality_score} = \text{avg_rating} * \text{answer_entropy_normalized}$. This can be recomputed nightly by a background job rather than on the request path.

7.4 Table: `user_answers`

Column	Type	Notes
<code>id</code>	PK	
<code>user_id</code>	FK → users	
<code>question_id</code>	FK → questions	
<code>selected_option_id</code>	FK → question_options	
<code>answered_at</code>	DateTime	

Unique constraint on (`user_id`, `question_id`) — a user answers each question once (re-answering, if ever allowed, would be an explicit “redo” feature, not implicit).

7.5 Table: `question_ratings`

Column	Type	Notes
<code>id</code>	PK	
<code>user_id</code>	FK → users	
<code>question_id</code>	FK → questions	
<code>rating</code>	Integer (1–5)	
<code>created_at</code>	DateTime	

7.6 Table: `compatibility_cache`

Precomputed/cached pairwise scores so shortlist reads are cheap:

Column	Type	Notes
<code>user_id_a</code>	FK → users	Always the smaller of the two IDs, to avoid duplicate rows
<code>user_id_b</code>	FK → users	
<code>overlap_count</code>	Integer	Number of matching answers
<code>total_compared</code>	Integer	Number of questions both have answered
<code>score</code>	Float	<code>overlap_count / total_compared</code> , optionally weighted by <code>quality_score</code>
<code>updated_at</code>	DateTime	

Composite index on `(user_id_a, score)` and `(user_id_b, score)` for fast shortlist queries.

7.7 Coin & Payment Tables

`coin_wallets`: `user_id` (PK/FK), `balance` (Integer), `updated_at`.

`coin_transactions`: `id`, `user_id`, `type` (Enum: `purchase`, `signup_bonus`, `promo_bonus`, `spend_chat_unlock`, `spend_breakdown`, `spend_instant_reveal`, `spend_boost`, `spend_reroll`, `refund`), `amount` (signed integer), `balance_after`, `reference_id` (nullable, e.g. conversation id this spend unlocked), `created_at`.

`iap_purchases`: `id`, `user_id`, `platform` (Enum: `ios`, `android`), `product_id`, `store_transaction_id` (unique), `raw_receipt`, `coins_credited`, `verified_at`, `created_at`.

7.8 Chat Tables

`conversations`: `id`, `user_id_a`, `user_id_b`, `unlocked_by_user_id` (who spent the coin), `created_at`, `last_message_at`. Unique constraint on `(user_id_a, user_id_b)`.

`messages`: `id`, `conversation_id`, `sender_id`, `content`, `sent_at`, `read_at` (nullable).

7.9 Trust & Safety Tables

`reports`: `id`, `reporter_id`, `reported_user_id`, `reason`, `details`, `created_at`, `status`.

`blocks: id, blocker_id, blocked_id, created_at.`

7.10 Misc

`devices: id, user_id, push_token, platform, last_seen_at.`

`notifications: id, user_id, type (Enum: new_match, new_message, coin_low_balance, ...), payload (JSON), read_at, created_at.`

8. API Reference (Target Surface)

All endpoints are prefixed conceptually by domain; actual routing follows the existing `APIRouter` -per-domain pattern already used in `router/`.

8.1 Auth (`router/auth.py`, extended)

Method	Path	Auth	Description
POST	<code>/signup</code>	none	Create account (existing)
POST	<code>/login</code>	none	Email+password login, returns access+refresh tokens (existing, extend for refresh)
POST	<code>/token</code>	none	OAuth2 password-flow, for Swagger/tooling (existing)
POST	<code>/auth/refresh</code>	refresh token	Issue new access token
POST	<code>/auth/logout</code>	access token	Revoke refresh token
POST	<code>/auth/verify-email</code>	none	Confirm email OTP/token
POST	<code>/auth/forgot-password</code>	none	Send reset link/OTP
POST	<code>/auth/reset-password</code>	reset token	Set new password

8.2 Profile

Method	Path	Auth	Description
GET	<code>/me</code>	user	Get own profile (existing <code>/user_profile</code> , renamed)
PATCH	<code>/me</code>	user	Update profile fields
POST	<code>/me/photos</code>	user	Upload a profile photo (returns signed upload URL or accepts multipart)
DELETE	<code>/me/photos/{photo_id}</code>	user	Remove a photo
DELETE	<code>/me</code>	user	Delete/deactivate account

8.3 Questions & Answers

Method	Path	Auth	Description
GET	/questions/session	user	Get the next batch (up to 10) of unanswered questions for this user
POST	/questions/{id}/answer	user	Submit an answer; triggers async recompute of this user's compatibility rows
POST	/questions/{id}/rate	user	Rate a question 1–5
GET	/questions/progress	user	Current session progress + live shortlist count teaser

8.4 Matching / Shortlist

Method	Path	Auth	Description
GET	/matches	user	Full revealed shortlist, ranked by score (requires session complete, or instant-reveal coin spend)
POST	/matches/reveal-now	user	Spend coins to reveal shortlist before finishing the session
GET	/matches/{user_id}/why	user	Compatibility breakdown (coin-gated)
POST	/matches/{user_id}/reroll	user	Skip to next-best match (coin-gated)
POST	/me/boost	user	Activate 24h profile boost (coin-gated)

8.5 Chat

Method	Path	Auth	Description
POST	/conversations	user	Start a conversation with a match (coin-gated: 1 coin)
GET	/conversations	user	List conversations (inbox)
GET	/conversations/{id}/messages	user	Message history (paginated)
POST	/conversations/{id}/messages	user	Send a message
WS	/ws/chat/{token}	user (token in query/header)	Realtime message stream

8.6 Coins & Payments

Method	Path	Auth	Description
GET	/coins/wallet	user	Current balance
GET	/coins/transactions	user	Transaction history
GET	/coins/bundles	none	List purchasable bundles + store product IDs
POST	/coins/purchases/verify	user	Submit App Store/Play receipt for server-side verification + credit
POST	/webhooks/apstore	store signature	Apple server-to-server notifications (refunds, renewals)
POST	/webhooks/playstore	store signature	Google real-time developer notifications

8.7 Trust & Safety

Method	Path	Auth	Description
POST	/users/{id}/report	user	Report a user
POST	/users/{id}/block	user	Block a user (removes from shortlists both ways)
GET	/users/blocked	user	List blocked users

8.8 Admin (internal use, separately authenticated)

Method	Path	Auth	Description
POST	/admin/questions	admin	Add a question (manual or reviewed AI-generated)
PATCH	/admin/questions/{id}	admin	Edit/deactivate a question
GET	/admin/reports	admin	Moderation queue
GET	/admin/metrics	admin	Core product metrics dashboard data (see §11)

9. Matching Algorithm

9.1 Basic Scoring

For any two users `A` and `B` who have both answered at least one common question:

```
overlap_count = count(questions where A.selected_option == B.selected_option)
total_compared = count(questions both A and B have answered)
raw_score = overlap_count / total_compared
```

9.2 Quality-Weighted Scoring (Phase 2)

Not all matching answers are equally meaningful — agreeing on a highly discriminating question (most people disagree) is a stronger compatibility signal than agreeing on a question almost everyone answers the same way. Weight each matching question by its `quality_score` (§7.3):

```
weighted_score = 
$$\frac{\sum(\text{quality\_score of each question where A and B agree})}{\sum(\text{quality\_score of each question both answered})}$$

```

9.3 Computation Strategy

- **Do not** compute pairwise scores across the entire user base on every request — this doesn't scale. Instead:
 1. When a user submits an answer, enqueue a background job (`recompute_compatibility(user_id)`).
 2. The job compares the user's answers only against other users **in the same city** (cold-start scoping, §3.1) and within the compatible gender-preference set, and upserts rows into `compatibility_cache` .
 3. Shortlist reads (`GET /matches` , and the live teaser count) are then simple indexed reads from `compatibility_cache` , never a live full-table scan.
- Recompute is naturally cheap early on (small city-scoped user base) and can be optimized later (e.g. via a nightly batch + incremental deltas) as volume grows.

9.4 Progressive Teaser Count

While a user is mid-session (has answered `k` of 10 questions), the “N people think like you” teaser is: count of same-city, preference-compatible users whose `overlap_count` on the `k` questions answered so far is above a minimum threshold (e.g. $\geq 70\%$). This is a cheap query against `user_answers` filtered to the current question set, not the full `compatibility_cache` , since the session is still in progress.

10. Mobile App: Screens & User Flows

10.1 Full Screen List

Onboarding & Auth

1. Splash screen
2. Welcome / value-proposition carousel (3 slides explaining the mechanic)
3. Sign Up (email + password)
4. Email verification (OTP or link confirmation)
5. Log In
6. Forgot Password

Profile Setup (mandatory before first question session)

1. Basic info (name, date of birth, gender, gender preference, city)
2. Photo upload (min 2, max 6)
3. Short bio / prompt answers (optional)

Question Flow

1. "How this works" intro screen (shown once, first session only)
2. Question card — one question at a time, large tappable option cards (would-you-rather style), progress bar showing `k / 10`
3. Optional post-answer rating (lightweight 1–5 star tap, dismissible, shown every ~3rd question to avoid fatigue)
4. Live teaser panel — "N people think like you" with blurred avatar stack, shown between questions
5. Session complete / reveal screen — CTA to view full shortlist, or upsell to "reveal early next time" (for future sessions)

Matches / Shortlist

1. Shortlist grid — ranked list/grid of revealed matches with photo, name, age, % compatibility ring
2. Match profile detail — full photos, bio, locked "Why we matched" section (coin unlock CTA)
3. Compatibility breakdown (unlocked) — question-by-question shared answers
4. Empty/low-density state — "Not many matches yet in your area — answer more questions or invite friends" (cold-start mitigation UX)

Chat

1. Conversations list (inbox)
2. Chat screen — message thread; if not yet unlocked, shows a locked banner with "Unlock chat — 1 coin" CTA instead of a text input
3. Icebreaker suggestion (auto-suggested opening line referencing a shared answer)

Coins / Monetization

1. Coin wallet / balance (accessible from a persistent header icon)
2. Buy coins — bundle selection (3 / 10 / 25), routes to native StoreKit/Play Billing sheet
3. Purchase confirmation
4. Transaction history
5. Contextual paywall/unlock bottom-sheet (reused component, triggered from chat unlock, breakdown unlock, instant reveal, boost, and re-roll actions)

Profile & Settings

1. My profile (view)
2. Edit profile / manage photos
3. Preferences (age range, gender preference, city/distance)

- 4. Question history (review previously answered questions — read-only)
- 5. Settings (notifications toggle, privacy, language, logout, delete account)
- 6. Block / report flow (from a user's profile or a chat's overflow menu)
- 7. Help & FAQ

Notifications

- 1. Notification center (new match, new message, low coin balance, etc.)

10.2 Primary Navigation Structure

Auth Stack (unauthenticated)

Splash → Welcome Carousel → [Sign Up | Log In] → Email Verify

Onboarding Stack (authenticated, not yet onboarded)

Basic Info → Photos → Bio → Question Intro → Question Session (10x) → Reveal

Main Tab Bar (authenticated + onboarded)

Shortlist	Chats	More Questions	Profile
(Matches)	(Inbox)	(answer more)	/ Settings

Persistent header icon (all tabs): Coin balance → Buy Coins

10.3 Key UX Principle

The reveal screen (14) is the emotional payoff of the core loop and should never be the point where the app asks for money outright — the coin CTA belongs *inside* the profile/chat screens that follow it (unlock chat, unlock breakdown), consistent with the monetization guardrail in §3.3.

11. Notifications Strategy

Trigger	Channel	Notes
New match revealed (session complete)	Push (FCM) + in-app	Drives return visits
New message received	Push (FCM) + in-app	Only if recipient not currently in that chat screen
Coin balance low (after a spend leaves < 1 coin)	In-app banner	Soft upsell, not push (avoid feeling spammy)
Someone rated your shared question highly / new question batch available	In-app only	Low priority
Account/report/moderation notices	Push + in-app	Required for trust & safety transparency

Push delivery goes through FCM; the backend's background worker (§4.2) is responsible for sending these after the triggering event (new message write, match reveal, etc.), using each user's `devices.push_token`.

12. AI-Generated Question Content Pipeline

12.1 Seeding Strategy

- Launch with a **hand-reviewed seed set of ~30 questions** (mix of `direct_compatibility` and `situational_values`, per §3.2), generated via an LLM prompt but manually approved before going live, to avoid embarrassing or ambiguous content at launch.
- Grow toward **100 questions** as the user base grows, adding batches of ~10–20 at a time based on which tags/categories are under-represented and which existing questions have the lowest `quality_score` (i.e., replace/deprioritize weak questions rather than only adding).

12.2 Generation Prompt Structure (example)

```
Generate {N} short, engaging "would-you-rather"-style situational questions for a
dating-compatibility app. Each question should:
- Present a realistic or lightly hypothetical situation
- Offer exactly {2-4} distinct response options that reveal differing values,
  personality, or behavior (avoid options where one is obviously "correct")
- Avoid explicit political, religious, or otherwise polarizing-for-its-own-sake content
- Be answerable in a few seconds, no more than 25 words for the prompt itself
- Be tagged with one category: "direct_compatibility" or "situational_values"
- Include 2-4 short tags describing the trait being probed (e.g. "risk-tolerance",
  "conflict-style", "ambition", "humor")

Return as JSON: [{ "prompt": ..., "options": [...], "category": ..., "tags": [...] }]
```

Generated batches go into a **review queue** (`is_active = false` until an admin approves via the admin endpoint in §8.8), not directly live.

12.3 Quality Feedback Loop

- User star ratings (§7.5) and computed `quality_score` (§7.3) feed back into which questions are served more often (`GET /questions/session` should weight selection toward higher-quality, currently-active questions once there's enough rating data).
- Once enough ratings exist, offer a **premium “popular questions only” mode**: users who prefer only answering highly-rated questions can opt into a filtered session (this was part of the original product idea — it's a natural coin-gated toggle, e.g. “Answer only ★4+ rated questions — 1 coin”).

13. Trust & Safety

- **Reporting & blocking** are first-class features from MVP (not an afterthought) — required by both app stores' guidelines for any app with user-generated content/messaging (Apple App Store Review Guideline 1.2, Google Play's User Generated Content policy).
 - **Photo moderation**: at minimum, a manual review queue for newly uploaded profile photos before a profile is visible to others; Phase 2 should add an automated NSFW/image-safety check (e.g. AWS Rekognition Moderation, or an open-source equivalent) to reduce manual review load.
 - **Message content**: basic profanity/abuse filtering server-side, plus a manual moderation queue fed by user reports.
 - **Age verification**: enforce 18+ at signup via `date_of_birth`; do not rely on self-attestation alone if budget allows a lightweight verification step later.
 - **Blocking effect**: a block must immediately remove both users from each other's `compatibility_cache` results and hide any existing conversation.
-

14. Non-Functional Requirements

Concern	Approach
Scalability	City-scoped matching keeps the hot working set small even as total users grow; <code>compatibility_cache</code> avoids recomputation on read; add read replicas / partition by city if a single city's data volume grows large
Performance	Target < 300ms p95 for <code>GET /matches</code> and <code>GET /questions/session</code> ; precomputation (§9.3) is what makes this achievable
Observability	Structured logging, request tracing (e.g. OpenTelemetry), and the <code>/admin/metrics</code> endpoint (§8.8) surfacing signups, session-completion rate, shortlist non-empty rate per city, coin conversion rate, and message-reply rate
Data privacy	Raw answers and messages are never exposed to any client except the involved user(s); compatibility breakdowns expose <i>which questions</i> matched, not the raw text of the other user's full answer history
Availability	Stateless FastAPI instances behind a load balancer; Postgres is the single source of truth; Redis failure should degrade gracefully (fall back to DB reads) rather than hard-fail
Testing	Unit tests for scoring logic (§9) and coin ledger transactions (§6) are highest priority given they directly affect money and trust

15. Development Roadmap

Phase 0 — Current State (done)

- FastAPI skeleton, PostgreSQL via SQLAlchemy, JWT auth (signup/login/token), `get_current_user` dependency, basic user listing.

Phase 1 — MVP (single-city launch)

- Extend `users` table with profile fields; profile setup screens.
- Seed 30 questions; question session flow (10 questions), progressive teaser count.
- Full shortlist reveal after session completion.
- Coin wallet with 3 free starting coins; 1-coin chat unlock; IAP purchase + server-side receipt verification for coin bundles.
- Basic 1:1 in-app chat (REST + polling acceptable for MVP; WebSocket upgrade can follow).
- Report/block; push notifications for new match and new message.
- Single-city gating for cold start.

Phase 2 — Depth & Retention

- Question rating system live; `quality_score`-driven curation; grow bank toward 100 questions.
- Compatibility breakdown (coin-gated); profile boost; re-roll.
- WebSocket-based realtime chat, read receipts.
- Email verification, password reset, refresh tokens, rate limiting.
- Basic photo moderation queue.
- Admin dashboard (question management, moderation, core metrics).

Phase 3 — Scale & Sophistication

- Multi-city expansion tooling; per-city launch playbook based on Phase 1 learnings.
- Quality-weighted matching algorithm (§9.2); possibly ML-based re-ranking from engagement signals.
- Social login (Sign in with Apple mandatory alongside any other social login) and/or phone OTP.
- Automated image moderation; expanded trust & safety tooling.
- “Popular questions only” premium mode (§12.3).

Phase 4 — Growth & Internationalization

- Localization (the existing `language` field on `users` is a head start).
 - Additional monetization surfaces (e.g. seasonal promos, referral bonuses with 30-day-expiring bonus coins per §6.5).
 - Infrastructure scale-out: read replicas, city-based partitioning, CDN for media.
-

16. Appendix

16.1 Glossary

- **Shortlist:** the set of other users whose answers overlap with the current user's above a threshold, at any point in a question session.
- **Reveal:** the moment a shortlist's real names/photos become visible (either after completing a 10-question session for free, or earlier via a coin-gated instant reveal).
- **Coin sink:** any action that costs coins.
- **Quality score:** a per-question metric combining average user rating and answer-distribution entropy, used to curate the question bank over time.

16.2 Suggested Repository Layout Going Forward

```
app/  
  main.py  
  database.py  
  auth.py  
  config.py          # centralize env var loading  
  models/  
    user.py  
    question.py  
    answer.py  
    compatibility.py  
    coin.py  
    chat.py  
    safety.py  
  schemas/  
    user.py  
    question.py  
    matching.py  
    coin.py  
    chat.py  
  services/  
    matching_service.py # scoring logic ($9)  
    coin_service.py     # ledger debits/credits ($6)  
    iap_service.py      # Apple/Google receipt verification ($6.4)  
    notification_service.py # FCM sending ($11)  
    question_gen_service.py # LLM batch generation ($12)  
  router/  
    auth.py  
    profile.py  
    questions.py  
    matching.py  
    chat.py  
    coins.py  
    safety.py  
    admin.py  
  workers/  
    tasks.py          # Celery/RQ tasks: recompute_compatibility, send_push, etc.
```

16.3 Key Decisions Log (from product discussion)

Decision	Rationale
PostgreSQL + FastAPI instead of Firebase-only	Compatibility matching needs relational queries and precomputation Firestore can't do natively; Firebase is repurposed for push notifications only
Coins instead of subscription	Usage pattern is bursty/curiosity-driven, not continuous — subscription mismatches behavior
3 free starting coins, not 1	Optimizes for “at least one good first conversation,” which drives long-term conversion better than minimizing giveaway
Full shortlist reveal is free after session completion; coins pay for <i>acceleration</i> and <i>extra insight</i>	Avoids the “pay to find out if it was worth it” dark pattern while still preserving the user’s original “premium on-the-spot check” idea as an optional upsell
Purchased coins never expire; promo/bonus coins expire in 30 days	Preserves goodwill on paid coins while keeping urgency on giveaways
Launch single-city/campus first	Mitigates cold-start empty-shortlist problem that has killed many dating app launches
Question bank starts at 30, grows toward 100	Matches the plan to validate before over-investing in content generation
Question rating system feeds a <code>quality_score</code> used for curation and a future paid “popular-only” mode	Turns user feedback directly into both content quality and a monetization lever